Αναφορά εργασίας

Ελευθέριος Παντζαρτζής 4344

1. Description of a greedy, efficient algorithm for the problem

The input of the algorithm is going to be x, k_0 and a 2D array S[n, 2]. S[i, 0] is for the stations xi and S[i, 1] for the fuel amounts ki. The idea is that we keep crossing each of the stations xi, of course while subtracting from our picked up fuel the distance to each station, while putting all the visited stations in a max-heap. Then, when at one point the fuel ends, we pop the max element (pretending that we had stopped at that station) from the heap and add it to our picked up fuel, and we also add one to the number of stops for refueling.

1. The target distance and an "infinite" fuel supply(huge number or inf in python) at the target are appended to the list of stations.
2. We sort S in an ascending order of x_i because each x_i represents the distance from the start, to ensure every station is visited sequentially.
3. A max-heap is initialized for tracking the max fuel amount of the visited stations.
4. Counters for the number of stops for refueling(refuels) and the previous station's distance from the start (prev) are initialized to 0.
5. The algorithm iterates through each station, processing them in the order they are encountered along the journey (having already been sorted by distance from the start). (so the "for location, capacity in stations" loop begins)
6. The fuel required to reach the current station is subtracted from the current amount of picked up fuel (totalfuel -= location - prev).
7. If the current fuel is insufficient to reach the current station(totalfuel <0), the algorithm refuels by using the

largest available fuel supplies from previously passed stations. This is done by popping the largest fuel supply from the heap and adding it to the current fuel (totalfuel += heapq.heappop(heap)). The number of refuels is incremented (refuels += 1).

8. This process is repeated until there is enough fuel to reach the current station or no more fuel supplies are available in the heap(it's empty).
9. If, after using all available fuel supplies from the heap, the fuel is still not enough to reach the current station(namely, if totalfuel < 0 after the while loop), the algorithm returns -1, indicating that the journey cannot be completed.
10. Else, if there is sufficient fuel to reach the current station, the fuel supply available at this station is added to the heap (heapq.heappush(heap, capacity)), and the prev position is updated to the current station's location.(prev = location).

Analysis of the correctness

The algorithm initializes an empty max-heap (max_heap) to store the available fuel supplies.

The target distance is added as the last station with an infinite fuel supply to ensure that the algorithm reaches the target distance x as a final station.

Because we sort S in increasing order means that in the for loop we process the stations in order of their distances from the starting point.

We keep crossing each of the stations xi, of while subtracting from our picked up fuel the distance to each station, while putting all the visited stations in a max-heap(heapq.heappush(heap, capacity)). Then when refueling is absolutely needed we ensure that we pick the maximum amount of fuel that can be picked by popping the max element from the heap. Namely, for each station, if the current fuel is insufficient to reach the next station

(totalfuel < 0), the algorithm refuels using the largest available fuel supply stored in the max_heap.
This is done until either sufficient fuel is available or no more fuel supplies are available in the heap.
If, after exhausting all available fuel supplies in the max_heap, the fuel is still insufficient to reach the next station (totalfuel < 0), the algorithm correctly returns -1 to indicate that reaching the target is not possible.
The loop continues until all stations are processed, including the final target station.
If the algorithm successfully processes all stations and reaches the target, it returns the total number of refuel stops made.

Analysis of the complexity

Time complexity:

n: number of stations

Sorting the Stations: O(nlogn)

Looping through the Stations: Each station is processed once, and each heap operation (push and pop) takes O(logn). Therefore, iterating through the n stations and performing heap operations takes O(nlogn).

So the overall time complexity is O(nlogn).

Space complexity:

The main occupier of space is the heap which can at worst grow to O(n) (we also have the 2D array nx2 S which is O(n)), so the overall space complexity is O(n).

2. ***Description of an efficient algorithm using dynamic programming***

Definition: OPT(i, j) = the least cost for the section of the string from position i to position j.

Subproblems:

The idea is to find the minimum cost to cut the string between the cut points. This leaves us with two subproblems each time and recursively they reduce down

to the base case when they can't be cut and become 0. Lastly, we add up all the values from each subproblem.

Case 1. If j - i <= 1

- OPT(i, j) = 0 since it's impossible to cut it further.

Case 2. For each k between cut[i] and cut[j], the minimum cost to cut the string from cut[i] to cut[j] is calculated as the sum of the cost to cut the string from cut[i] to cut[k] and from cut[k] to cut[j], plus the cost to make the cut in that subinterval.

- OPT(i, j)=$\min_{i<k<j}$(OPT(i, k)+OPT(k, j)+ cut[j] − cut[i])

($\min_{i<k<j}$ here refers to the minimum across the for loop: for k in range(i + 1, j), not including j)

Bellman Equation.

$$OPT(i,j) = \begin{cases} 0 \; if \; j-i \leq 1 \\ min: i<k<j \; (OPT(i,\ k) + OPT(k,\ j) + \text{cut[j]} - \text{cut[i]}) \; if \; j-i > 1 \end{cases}$$

If we wanted to calculate the order by which we get the minimum cost then we use memoization and then backtracking. For that we need to track the cuts using an mxm array cut_sequence[i, j] to then reconstruct the final sequence of cuts. In pseudocode:

STRCOST(i, j)

IF (j - i <= 1)

    RETURN 0.

IF (memo[i,j]=inf)//initialized memo with inf, like python inf

    FOR k=i+1 TO j-1 //including j-1

        cost1 ← strcost(i, k) + strcost(k, j) + cuts[j] - cuts[i]. //strcost is the name of our recursive function

IF (cost1 < memo[i, j])

memo[i, j] ← cost1.

cut_sequence[i, j] ← k. //we keep the k

RETURN memo[i, j].

The calculation of the solution sequence is done with a second pass using the function FIND-SOLUTION(i, j) by backtracking. In pseudocode:

FIND-SOLUTION(i, j)

IF (j - i<=1)

RETURN ∅. //return empty array

k ← cut_sequence[i, j].

RETURN FIND-SOLUTION(i, k) ∪ {cut[k]} ∪ FIND-SOLUTION(k, j).

Analysis of the correctness

A memoization table memo mxm is initialized to store the minimum cost for each cut. Initially, all values are set to infinity.(inf)

Base Case: IF (j - i <= 1)

RETURN 0.

Since a string of that length can't be cut.

Recursion: IF (memo[i,j]=inf)

FOR k=i+1 TO j-1 //including j-1

memo[i, j] ← MIN(memo[i, j], strcost(i, k) + strcost(k, j) + cut[j] - cut[i]). //strcost is the name of our recursive function

If the cost for the subproblem (i, j) is not yet computed (therefore it's still infinity), the function calculates it by considering all possible cuts k between i and j.

For each possible cut k, it computes the total cost as the sum of the costs of the left and right segments plus the cost of cutting at k (which is cut[j] - cut[i]) and the minimum of these values is stored in memo[i, j].

The results of subproblems are stored in memo to avoid redundant calculations and improve efficiency.

Analysis of the complexity

Time complexity:

m: number of cuts to be made

The function OTP(i, j) potentially calls OTP(i, k) and OTP(k, j) for every possible k between i and j. And since there are $O(m^2)$ (like the number of nodes in a binary tree) possible subproblems the overall time complexity is $O(m^3)$.

Space complexity:

The main occupier of space is the memo table which requires $O(m^2)$ space, so the overall space complexity is $O(m^2)$.

Additionally, here's a small analysis for the correctness and complexity for the function FIND-SOLUTION when backtracking:

Analysis of the correctness

Base Case: IF (j - i<=1)

RETURN ∅. //return empty array

This base case ensures that when there are no cuts to be made (namely, the difference between i and j is 1 or less), the

function returns an empty list. This is correct because no cuts are needed in this case.

k ← cut_sequence[i, j].

Here, k is the optimal cut point between i and j that we determined during the dynamic programming phase when we filled the cut_sequence table.

Recursion: RETURN FIND-SOLUTION(i, k) ∪ {cut[k]} ∪ FIND-SOLUTION(k, j).

The function recursively calls itself to find the sequence of cuts for the left segment (i to k) and for the right segment (k to j). It combines these results with the current optimal cut cut[k]. This is correct since all segments are considered and the sequence of cuts that minimizes the cost is correctly constructed by combining the results from both subproblems and adding the current cut.

Analysis of the complexity

Time complexity:

m: number of cuts to be made

Each recursive call processes one cut and there are m cuts so in the worst case scenario we have O(m).

Space complexity:

The space complexity for storing the memo and cut_sequence tables is $O(m^2)$. There's also the recursion stack of the function FIND-SOLUTION which in the worst case is O(m), so the overall time complexity is $O(m^2)$.

3. (a) ***Description of an efficient algorithm using dynamic programming***

Definition: OPT(i, j) = the maximum total value that can be obtained by selecting cells up to row i with cell(i, j) included.
Subproblems:

- Calculating the Maximum Value for Each Cell: Starting from the first row, we calculate the maximum possible value for each cell, taking into account the values of the cells in the previous rows and the constraints of the problem.
- Calculating Maximum Total Value: After calculating the maximum values for each cell, we find the maximum total value that can be achieved by selecting one cell from each row. This value is the maximum value among the values calculated for the last cell of the chessboard.
- Retrieving the Optimal Solution: After calculating the maximum total value, we identify the cells that are selected to achieve this value. These cells correspond to the cells selected from each row that meet the constraints of the problem.

Case 1. If i = n:

- OPT(i, j) = 0 since we reached the last row of the chessboard.

Case 2. Calculate the maximum total value obtained by selecting cells up to row i−1, excluding the column j of the current row. This ensures that we don't select cells from the same column in consecutive rows.

- OPT(i, j)= $\max_{k \neq j}$ OPT(i-1,k)+V(i,j)

($\max_{k \neq j}$ here refers to the max across the for loop: for k in range(n), not including n)
Bellman Equation.

$$OPT(i,j) = \begin{cases} V[i,j] \; if \; i = 0 \\ max: k \neq j \left(OPT(i-1,k) + V(i,j)\right) \; if \; i > 0 \end{cases}$$

If we wanted to calculate the n cells that maximize the value then we use memoization and then backtracking. In pseudocode:

```
n ← len(V)
initialize memo nxn with empty values
CHESSBOARD(row, col):
IF (row = 0)
    RETURN V[0, col].
IF (memo[row, col] is not Empty)
    RETURN memo[row, col]. //to reduce the number of unnecessary iterations


max ← 0.
FOR k=1 TO n-1 //including n-1
    IF (k != col)
        max = MAX(max, CHESSBOARD (row - 1, k)).
memo[row, col] = max + V[row, col].
RETURN memo[row, col].
FOR j=0 TO n-1 //including n-1
    total = CHESSBOARD(n - 1, j). //call on the last row
    max_total = MAX(max_total, total). //to bring everything together
```

The calculation of the solution sequence is done with a second pass using the function FIND-SOLUTION(i, j) by backtracking. In pseudocode:

```
FIND-SOLUTION(i, j)
FOR i=n-1 TO 0
```

```
FOR j=0 TO n
    IF (CHESSBOARD(i, j) = current_value)
        selected_cells.APPEND((i, j)).
        current_value ← current_value - V[i][j].
        BREAK.
RETURN selected_cells.
```

Analysis of the correctness

- The algorithm aims to find the combination of cells that yields the maximum total value while adhering to the constraints of selecting one cell from each row and not selecting two cells from the same column in consecutive rows. By employing dynamic programming with memoization, it recursively explores all possible selections and efficiently identifies the optimal solution.
- After finding the maximum total value, the algorithm backtracks to determine which cells were selected to achieve this value. It iterates through the memoization table from the bottom row to the top row, identifying the cells that contribute to the current total value. The selected cells are then validated to ensure they meet the constraints specified in the problem statement.
- The backtracking process ensures that the selected cells are valid and contribute to the maximum total value. It traverses the memoization table in reverse order, confirming that each selected cell's value corresponds to the current total value being tracked. If a cell is selected, its value is added to the current total value, and this continues until the top row is reached.
- The algorithm satisfies the constraints of selecting one cell from each row and avoiding the selection of two cells from the same column in consecutive rows. These constraints are enforced during both the dynamic

programming computation and the backtracking process, ensuring that the solution adheres to the problem requirements.

(b) ***Analysis of the complexity***

Dynamic Programming:

Time complexity:

The function CHESSBOARD(row, col) is called for each cell in the matrix. Since the memoization table stores results to avoid redundant calculations, each cell is computed at most once, so $O(n^2)$. The computation for each cell involves iterating over all columns of the previous row. This results in an O(n) time complexity for each cell.

So the overall time complexity is $O(n^3)$.

Space complexity:

The main occupier of space is the memoization table memo with dimensions nxn so the overall complexity is $O(n^2)$.

Greedy approach:

Time complexity:

Its complexity is $O(n^2 \log n)$ in the worst case. This is due to the need to sort the cells of each row by their value, which takes O(nlogn) time for each row.

Space complexity:

The main occupier of space is the array V:$O(n^2)$ so the overall space complexity is $O(n^2)$.

(c') Correctness of the greedy algorithm

Regarding the correctness of the greedy algorithm, it is not guaranteed to always find the optimal solution. In some cases it can provide a solution with a lower overall value than the

dynamic programming algorithm. For example, if there are rows with a very large value in individual cells, the greedy algorithm may select those cells first and exclude cells with a smaller value that would be important to the overall value:

[1,10,3]

[1,9,3]

[1,2,8]